Spring Boot Security + Thymeleaf MVC 學習文件

> **適用對象**：已了解 Spring Boot 基本概念，想學習如何整合 Spring Security 與 Thymeleaf 實作登入/登出功能的初中級學習者。

目錄

1. 整體架構概覽
2. 專案依賴設定
3. 資料庫與 JPA 設定
4. 資料模型 (Model)
5. Repository 介面
6. 安全設定 (SecurityConfig)
7. 自訂使用者服務 (CustomUserDetailsService)
8. Controller 控制器
9. 資料初始化 (DataInitializer)
10. Thymeleaf 頁面
11. 完整請求流程圖
12. 常見陷阱與注意事項

整體架構概覽

這個專案示範如何使用 **Spring Boot + Spring Security + Thymeleaf** 建立一個完整的 MVC 登入/登出系統。

```
HTTP 請求
    │
    ▼
SpringSecurityConfig
    ├── SecurityFilterChain   ← 決定哪些路徑需要認證
    ├── PasswordEncoder       ← BCrypt 密碼比對
    └── AuthenticationManager
            │
            ▼
    CustomUserDetailsService
            │  loadUserByUsername(usernameOrEmail)
            ▼
    UserRepository
            │  findByUsernameOrEmail(...)
            ▼
    資料庫 (users / roles 資料表)
```

核心概念快覽

元件	角色	關鍵技術術語
`SecurityConfig`	設定安全規則、登入/登出行為	`SecurityFilterChain`
`CustomUserDetailsService`	告訴 Spring Security 如何找到使用者	`UserDetailsService`
`User` / `Role`	資料庫對應的實體類別	`@Entity`, `@ManyToMany`
`UserRepository`	資料庫查詢介面	`JpaRepository`
`AuthController`	頁面路由，回傳 Thymeleaf 模板名稱	`@Controller`, `@GetMapping`
`DataInitializer`	應用啟動時自動建立初始資料	`CommandLineRunner`

專案依賴設定

在 `pom.xml` 中需要加入以下依賴（`dependency`，相依套件）：

```
<!-- Spring Boot Web (MVC) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Spring Security -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Thymeleaf 模板引擎 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- Thymeleaf + Spring Security 整合（讓 HTML 可使用 sec:authorize） -->
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>

<!-- Spring Data JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
```

```
</dependency>

<!-- Lombok (簡化 getter/setter/constructor) -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

資料庫與 JPA 設定

位置：`src/main/resources/application.properties`

```
# 資料庫連線設定
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=1234
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA 設定
spring.jpa.hibernate.ddl-auto=update       # 自動更新資料表結構（不刪資料）
spring.jpa.show-sql=true                   # 在 console 印出 SQL 語句
spring.jpa.properties.hibernate.format_sql=true               # SQL 格式化排版
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

重點說明

設定	說明
`ddl-auto=update`	每次啟動時，若資料表不存在就建立，欄位有變就新增，**不刪除資料**
`ddl-auto=create`	每次啟動都重建資料表（**會清空資料**，只適合開發初期）
`ddl-auto=validate`	只驗證結構是否符合，不修改（正式環境推薦）

資料模型 (Model)

User 實體類別

```
@Getter
@Setter
@NoArgsConstructor          // Lombok：自動產生無參數建構子
@Entity                     // JPA：標示這是資料庫對應的實體
@Table(name = "users")      // 對應資料庫的 "users" 資料表
public class User {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // 自動遞增 ID
    private Long id;

    @Column(nullable = false, unique = true)  // 非空、不重複
    private String username;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;   // ⚠️ 儲存 BCrypt 加密後的密碼，絕對不能明文存入

    // 多對多關聯：一個使用者可以有多個角色
    @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
    @JoinTable(
        name = "users_roles",                                    // 中間關聯表名
稱
        joinColumns        = @JoinColumn(name = "user_id"),      // 本表外鍵
        inverseJoinColumns = @JoinColumn(name = "role_id")       // 對方外鍵
    )
    private Set<Role> roles = new HashSet<>();
}
```

逐行解析：`@ManyToMany` 多對多關聯

```
User ── users_roles ── Role
  1          N:M         N
```

- `@ManyToMany`：宣告多對多關係（一個使用者有多個角色，一個角色也可以屬於多個使用者）
- `fetch = FetchType.EAGER`：查詢 User 時，**立即**同時載入所有關聯的 Role（適合角色數量少的情境）
- `@JoinTable`：指定中間關聯表 `users_roles` 的欄位對應

> **常見陷阱**：`FetchType.EAGER` 在角色數量多時會造成 N+1 查詢問題，正式環境建議改為 `FetchType.LAZY`。

Role 實體類別

```
@Getter
@Setter
@NoArgsConstructor
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, unique = true)
    private String name;   // 例如 "ROLE_ADMIN", "ROLE_USER"
}
```

> **重要規範**：Spring Security 的角色名稱**必須以 `ROLE_` 開頭**。 例如：`ROLE_ADMIN`、`ROLE_USER`。在 `hasRole('ADMIN')` 中 Spring 會自動加上前綴。

Repository 介面

Repository（儲存庫）是 JPA 的資料存取層，繼承 `JpaRepository` 即可免費獲得 CRUD 方法。

```
public interface UserRepository extends JpaRepository<User, Long> {
    // Spring Data JPA 依照方法名稱自動產生 SQL：

    Optional<User> findByUsernameOrEmail(String username, String email);
    //  → SELECT * FROM users WHERE username = ? OR email = ?

    Optional<User> findByUsername(String username);
    //  → SELECT * FROM users WHERE username = ?

    boolean existsByUsername(String username);
    //  → SELECT COUNT(*) > 0 FROM users WHERE username = ?

    boolean existsByEmail(String email);
    //  → SELECT COUNT(*) > 0 FROM users WHERE email = ?
}
```

```
public interface RoleRepository extends JpaRepository<Role, Long> {
    Optional<Role> findByName(String name);
    //  → SELECT * FROM roles WHERE name = ?
}
```

方法命名規則

前綴	功能	範例
`findBy`	查詢單筆，回傳 `Optional<T>`	`findByUsername(...)`
`findAllBy`	查詢多筆，回傳 `List<T>`	`findAllByRole(...)`
`existsBy`	是否存在，回傳 `boolean`	`existsByEmail(...)`
`deleteBy`	刪除，回傳 `void`	`deleteByUsername(...)`
`Or` / `And`	組合條件	`findByUsernameOrEmail(...)`

安全設定 (SecurityConfig)

這是整個安全機制的核心，告訴 Spring Security「誰可以存取哪些頁面」以及「如何處理登入/登出」。

```
@Configuration          // 宣告這是設定類別 (Configuration class)
@EnableWebSecurity      // 啟用 Spring Security 的 Web 安全功能
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // ① CSRF 保護
            .csrf(csrf -> csrf.disable())
            // ⚠️ 教學用途才關閉 CSRF，正式環境 Thymeleaf 會自動處理，
            //    可改為 .csrf(Customizer.withDefaults()) 保留保護

            // ② 請求授權規則
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/register").permitAll()  // 這些路徑不
需登入
                .anyRequest().authenticated()                        // 其他全部需
要登入
            )

            // ③ 表單登入設定
            .formLogin(form -> form
                .loginPage("/login")                    // 自訂登入頁面的路徑
                .loginProcessingUrl("/login")           // 表單 POST 送出的目標
 (Spring Security 攔截)
                .defaultSuccessUrl("/welcome", true)    // 登入成功後跳轉 (true =
強制跳轉，不管之前在哪頁)
                .failureUrl("/login?error=true")        // 登入失敗後跳轉
                .permitAll()
            )

            // ④ 登出設定
            .logout(logout -> logout
                .logoutRequestMatcher(new AntPathRequestMatcher("/logout"))  // 登
出路徑 (預設是 POST /logout)
                .logoutSuccessUrl("/login?logout=true")   // 登出成功後跳轉
                .invalidateHttpSession(true)              // 明確清除 Session (會
話)
                .deleteCookies("JSESSIONID")              // 清除 Cookie
                .permitAll()
            );

        return http.build();
    }

    // 密碼加密器：使用 BCrypt 演算法
    @Bean
    public PasswordEncoder passwordEncoder() {
```

```
        return new BCryptPasswordEncoder();
        // BCrypt 每次加密結果都不同（加了 salt），但 matches() 仍能驗證
    }
}
```

Security Filter Chain 流程

```
HTTP 請求
    │
    ▼
① UsernamePasswordAuthenticationFilter
    │   (攔截 POST /login，取出帳號密碼)
    ▼
② AuthenticationManager
    │   (呼叫 CustomUserDetailsService 從 DB 載入使用者)
    ▼
③ PasswordEncoder.matches()
    │   (比較輸入密碼與 BCrypt 雜湊值)
    ▼
④ 成功 → 跳轉 /welcome
   失敗 → 跳轉 /login?error=true
```

自訂使用者服務 (CustomUserDetailsService)

Spring Security 需要知道「如何從資料庫找到使用者」，這個介面就是橋樑。

```
@Service
@AllArgsConstructor   // Lombok：自動產生包含所有 final 欄位的建構子（相當於
@Autowired)
public class CustomUserDetailsService implements UserDetailsService {
    //                                      ^^^^^^^^^^^^^^^^^^^^^^^^
    //                                 實作 Spring Security 提供的介面

    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String usernameOrEmail)
            throws UsernameNotFoundException {

        // 1. 從資料庫查詢使用者（同時支援帳號或 Email 登入）
        User user = userRepository.findByUsernameOrEmail(usernameOrEmail,
usernameOrEmail)
                .orElseThrow(() ->
                    new UsernameNotFoundException("User not exists by Username or
Email"));
                    //                ↑ 找不到使用者時拋出例外，Spring Security 會處
理
```

```
        // 2. 將 Role 轉換為 Spring Security 認識的 GrantedAuthority 格式
        Set<GrantedAuthority> authorities = user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority(role.getName()))
                //                                      ↑ 例如 "ROLE_ADMIN"
                .collect(Collectors.toSet());

        // 3. 回傳 Spring Security 的 UserDetails 物件
        //    使用完整路徑避免與自訂 User 類別命名衝突
        return new org.springframework.security.core.userdetails.User(
                usernameOrEmail,    // 登入用帳號
                user.getPassword(), // BCrypt 加密後的密碼
                authorities         // 角色/權限清單
        );
    }
}
```

關鍵概念：`UserDetailsService` 介面

```
UserDetailsService (介面)
    └── loadUserByUsername(String username): UserDetails
                                                ↑
                                          包含: username, password, authorities
```

Spring Security 在驗證登入時，會自動偵測並使用你提供的 `UserDetailsService` 實作。**只要宣告為 `@Service`，Spring Security 就會自動找到它，不需要額外設定。**

Controller 控制器

```
@Controller   // ← 注意: 不是 @RestController (因為要回傳 HTML 頁面, 不是 JSON)
public class AuthController {

    // GET /login → 渲染 templates/login.html
    @GetMapping("/login")
    public String loginPage() {
        return "login";    // Thymeleaf 會到 templates/ 資料夾找 login.html
    }

    // GET /welcome → 渲染 templates/welcome.html
    @GetMapping("/welcome")
    public String welcome() {
        return "welcome";
    }
}
```

`@Controller` vs `@RestController` 差異

注解	回傳型別	適用場景
`@Controller`	模板名稱（String）→ HTML 頁面	MVC + Thymeleaf
`@RestController`	物件 → JSON 字串	REST API

> **重要**：登入的 `POST /login` 不需要在 Controller 中定義！Spring Security 會自動攔截並處理，這是框架的特性。

資料初始化 (DataInitializer)

```
@Component
@AllArgsConstructor
public class DataInitializer implements CommandLineRunner {
//                                        ^^^^^^^^^^^^^^^^
//                  實作此介面的 run() 方法，會在應用啟動後自動執行一次

    private final RoleRepository roleRepository;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    @Transactional   // 確保資料庫操作在同一個交易（transaction）中完成
    public void run(String... args) {

        // 1. 建立角色（使用 orElseGet 避免重複建立）
        Role adminRole = roleRepository.findByName("ROLE_ADMIN")
                .orElseGet(() -> {
                    Role r = new Role();
                    r.setName("ROLE_ADMIN");
                    return roleRepository.save(r);
                });
        // ↑ 如果 "ROLE_ADMIN" 已存在就直接用，否則建立新的

        Role userRole = roleRepository.findByName("ROLE_USER")
                .orElseGet(() -> {
                    Role r = new Role();
                    r.setName("ROLE_USER");
                    return roleRepository.save(r);
                });

        // 2. 建立管理員帳號（檢查是否已存在，避免重複）
        if (!userRepository.existsByUsername("admin")) {
            User admin = new User();
            admin.setUsername("admin");
            admin.setEmail("admin@example.com");
            admin.setPassword(passwordEncoder.encode("admin123"));  // ← BCrypt 加
密
            admin.setRoles(Set.of(adminRole));
            userRepository.save(admin);
            System.out.println("✅ 預設管理員帳號已建立：admin / admin123");
```

```
        }
    }
}
```

Thymeleaf 頁面

login.html — 登入頁面

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<!--        ↑ 宣告 Thymeleaf 命名空間，讓 th:* 屬性生效 -->
<head>
    <meta charset="UTF-8">
    <title>登入</title>
</head>
<body>
    <h2>登入系統</h2>

    <!-- 登入失敗訊息：URL 有 ?error 參數時顯示 -->
    <div th:if="${param.error}" style="color:red;">
        帳號或密碼錯誤，請重試。
    </div>

    <!-- 登出成功訊息：URL 有 ?logout 參數時顯示 -->
    <div th:if="${param.logout}" style="color:green;">
        已成功登出。
    </div>

    <!--
        th:action="@{/login}" → Thymeleaf 產生正確的相對路徑 URL
        method="post"         → Spring Security 只攔截 POST /login
        若 CSRF 未關閉，Thymeleaf 會自動插入 <input type="hidden" name="_csrf"
value="...">
    -->
    <form th:action="@{/login}" method="post">
        <div>
            <label>帳號或 Email：</label>
            <input type="text" name="username" required />
            <!--              ↑ name 必須是 "username"，Spring Security 預設讀取此欄
位 -->
        </div>
        <div>
            <label>密碼：</label>
            <input type="password" name="password" required />
            <!--                  ↑ name 必須是 "password" -->
        </div>
        <button type="submit">登入</button>
    </form>
```

```
</body>
</html>
```

---

welcome.html — 歡迎頁面

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<!--         ↑ 宣告 Spring Security 的 Thymeleaf 擴充命名空間 -->
<head>
    <meta charset="UTF-8">
    <title>歡迎</title>
</head>
<body>
    <!-- sec:authentication="name" → 顯示當前登入的使用者名稱 -->
    <h2>歡迎, <span sec:authentication="name"></span>! </h2>

    <!-- sec:authentication="principal.authorities" → 顯示使用者角色清單 -->
    <p>你的角色:
        <span sec:authentication="principal.authorities"></span>
    </p>

    <!-- sec:authorize="hasRole('ADMIN')" → 只有 ADMIN 角色才能看到此區塊 -->
    <div sec:authorize="hasRole('ADMIN')">
        <a href="/admin">進入管理頁面</a>
    </div>

    <!-- th:href="@{/logout}" → Thymeleaf 產生 /logout 連結 -->
    <a th:href="@{/logout}">登出</a>
</body>
</html>
```

Thymeleaf Security 常用屬性

屬性	說明	範例
`sec:authorize`	條件顯示區塊	`sec:authorize="hasRole('ADMIN')"`
`sec:authentication`	取得認證資訊	`sec:authentication="name"`
`th:if`	條件顯示	`th:if="${param.error}"`
`th:action`	生成表單 action URL	`th:action="@{/login}"`
`th:href`	生成連結 URL	`th:href="@{/logout}"`

完整請求流程圖

登入流程

```
① 使用者訪問受保護頁面（例如 /welcome）
    │
    ▼
② Spring Security 偵測未登入 → 自動重導向 /login
    │
    ▼
③ AuthController.loginPage() 回傳 login.html
    │
    ▼
④ 使用者填寫帳號密碼，POST /login
    │
    ▼
⑤ Spring Security 攔截 POST /login
    │
    ▼
⑥ CustomUserDetailsService.loadUserByUsername() 查詢資料庫
    │
    ├─ 找不到使用者 → 跳轉 /login?error=true
    │
    ▼
⑦ BCryptPasswordEncoder.matches(輸入密碼, DB密碼)
    │
    ├─ 密碼不符 → 跳轉 /login?error=true
    │
    ▼
⑧ 登入成功，建立 Session → 跳轉 /welcome
```

登出流程

```
① 使用者點擊登出連結 GET /logout
    │
    ▼
② Spring Security 攔截 /logout
    │
    ▼
③ 清除 Session（invalidateHttpSession）
   清除 Cookie（deleteCookies JSESSIONID）
    │
    ▼
④ 跳轉 /login?logout=true
    │
    ▼
⑤ login.html 顯示「已成功登出」訊息
```

常見陷阱與注意事項

❌ 陷阱 1：表單欄位名稱錯誤

```
<!-- ❌ 錯誤：Spring Security 找不到帳號密碼 -->
<input type="text" name="user" />
<input type="password" name="pass" />

<!-- ✅ 正確：預設必須是 username 和 password -->
<input type="text" name="username" />
<input type="password" name="password" />
```

> 如果你的欄位名稱不同，需要在 `formLogin()` 中加上：
>
> ```
> .usernameParameter("user")
> .passwordParameter("pass")
> ```

❌ 陷阱 2：忘記宣告 Thymeleaf Security 命名空間

```
<!-- ❌ 錯誤：沒有宣告 xmlns:sec，sec:authorize 不生效 -->
<html xmlns:th="http://www.thymeleaf.org">

<!-- ✅ 正確：需同時宣告兩個命名空間 -->
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

❌ 陷阱 3：密碼明文存入資料庫

```
// ❌ 錯誤：絕對不能這樣做！
user.setPassword("admin123");

// ✅ 正確：一定要用 PasswordEncoder 加密後再存入
user.setPassword(passwordEncoder.encode("admin123"));
```

❌ 陷阱 4：角色名稱沒有 `ROLE_` 前綴

```
// ❌ 錯誤：
role.setName("ADMIN");

// ✅ 正確：Spring Security 要求角色名稱以 ROLE_ 開頭
role.setName("ROLE_ADMIN");
```

❌ 陷阱 5：CSRF 設定問題

```
// ⚠️ 注意：正式環境不要關閉 CSRF
.csrf(csrf -> csrf.disable())   // 只適合教學/API 環境

// ✅ 使用 Thymeleaf 時，保留預設 CSRF 保護即可
// Thymeleaf 會自動在 <form> 中插入 _csrf 隱藏欄位
// .csrf(Customizer.withDefaults())  ← 預設值，可省略
```

學習筆記摘要

核心知識點

```
Spring Security MVC 知識樹
│
├── 安全設定 (SecurityConfig)
│   ├── SecurityFilterChain        ← 安全過濾器鏈，所有請求的入口
│   ├── authorizeHttpRequests()    ← 設定哪些路徑需要登入
│   ├── formLogin()                ← 表單登入相關設定
│   └── logout()                   ← 登出相關設定
│
├── 使用者驗證
│   ├── UserDetailsService         ← 介面：告訴 Spring 如何載入使用者
│   ├── UserDetails                ← 介面：Spring Security 認識的使用者格式
│   ├── GrantedAuthority           ← 介面：代表一個角色/權限
│   └── BCryptPasswordEncoder      ← 密碼加密器
│
├── 資料層
│   ├── @Entity / @Table           ← JPA 實體映射
│   ├── @ManyToMany / @JoinTable   ← 多對多關聯
│   └── JpaRepository              ← CRUD + 自訂查詢方法
│
└── 前端模板 (Thymeleaf)
    ├── th:action / th:href        ← URL 生成
    ├── th:if="${param.xxx}"       ← URL 參數條件判斷
    ├── sec:authorize              ← 角色條件顯示
    └── sec:authentication="name"  ← 顯示登入使用者資訊
```

需要記住的 Bean 自動連線關係

```
SecurityConfig
    └── 使用 PasswordEncoder → DataInitializer 也注入

CustomUserDetailsService
    └── Spring Security 自動偵測並使用
    └── 注入 UserRepository → 查詢 MySQL
```

```
DataInitializer (CommandLineRunner)
    └── 啟動時自動執行 run()
    └── 建立初始角色和帳號
```